

# Contents

|                                                                                        |          |
|----------------------------------------------------------------------------------------|----------|
| <b>guide Universal and Transferable Adversarial Attacks on Aligned Language Models</b> | <b>1</b> |
| 0. 安全边界和导读定位                                                                           | 1        |
| 1. 论文地位和当时语境                                                                           | 2        |
| 2. 原论文结构地图                                                                             | 2        |
| 3. 核心概念                                                                                | 3        |
| 4. 威胁模型和任务形状                                                                           | 3        |
| 5. 数学目标函数                                                                              | 4        |
| 6. Algorithm 1: GCG 怎么工作                                                               | 5        |
| 7. Algorithm 2: universal multi-prompt and multi-model                                 | 6        |
| 8. 和 soft prompt/embedding optimization 的区别                                            | 7        |
| 9. 论文的 benchmark: AdvBench                                                             | 7        |
| 10. 实验证据链: white-box                                                                   | 7        |
| 11. 实验证据链: black-box transfer                                                          | 8        |
| 12. Figure 6: 过拟合和迁移                                                                   | 9        |
| 13. 伦理讨论和 disclosure                                                                   | 9        |
| 14. 方法图: 从请求到 ASR                                                                      | 9        |
| 15. 代码样例: safe toy GCG                                                                 | 10       |
| 16. 和本仓库代码的连接                                                                          | 11       |
| 17. 一个 30-60 分钟本地实验                                                                    | 11       |
| 18. 论文没有证明什么                                                                           | 11       |
| 19. 对今天安全学习的意义                                                                         | 12       |
| 20. 用 AI agent 正确学习这篇论文                                                                | 12       |
| 21. 闭卷掌握检查                                                                             | 12       |
| 22. 一句话总结                                                                              | 13       |

## guide Universal and Transferable Adversarial Attacks on Aligned Language Models

原论文: Universal and Transferable Adversarial Attacks on Aligned Language Models

本地原文 PDF 目录: `learning/red-team-jailbreak/paper/`

文件名: `01_universal_transferable_adversarial_attacks_llms.pdf`

作者: Andy Zou, Zifan Wang, Nicholas Carlini, Milad Nasr, J. Zico Kolter, Matt Fredrikson

arXiv 版本: 2023 初版, 2023-12-20 v2

本地代码: `learning/red-team-jailbreak/src/`

### 0. 安全边界和导读定位

这篇论文是 LLM safety/red-team 方向的经典论文。它提出的 Greedy Coordinate Gradient, 简称 GCG, 把 jailbreak 从人工提示词技巧推进到可优化、可评测、可迁移的 adversarial suffix 搜索问题。

这份导读只用于防御研究、评测理解和安全工程学习。为了避免把 guide 变成可操作的攻击手册, 下面会遵守三个边界:

- 不复现原文里的危险请求或完整模型输出。

- 不提供真实可复用的 adversarial suffix。
- 代码样例只使用 toy target 和 harmless SAFE\_ACK 目标，帮助你理解离散优化结构。

你读这篇论文时要抓住一句话：

```
manual jailbreak:
  human invents a prompt trick
  -> brittle, hard to scale, hard to benchmark
```

```
GCG-style attack:
  turn suffix search into discrete optimization
  -> use gradients to propose token replacements
  -> use forward passes to verify candidates
  -> train one suffix across prompts and models
  -> test transfer to unseen models
```

这篇论文真正值得学的是安全评测的思维方式：如果一个 alignment 方法只对自然语言、人类手写 attack 有鲁棒性，不代表它对自动化 adversarial optimization 有鲁棒性。

## 1. 论文地位和当时语境

在这篇论文之前，LLM jailbreak 大多有两个来源：

- 人工构造：人写角色扮演、前缀诱导、多轮绕行等 prompt。
- 早期自动 prompt search：尝试用梯度或 soft prompt 做攻击，但在 aligned chatbot 上成功率有限。

人工 jailbreak 的问题是：

- 依赖人的创意，很难系统覆盖风险空间。
- prompt 经常脆弱，模型稍微升级或模板变化就失效。
- 很难解释一个 jailbreak 是偶然技巧，还是暴露了模型的结构性弱点。

传统 NLP adversarial attack 的问题是：

- 文本是离散 token，不像图像可以直接对像素做小连续扰动。
- 对分类器的 attack 通常要求语义不变，这在 LLM 生成任务里不容易定义。
- soft prompt 可以连续优化，但真实 public chatbot 接口通常只能接收离散文本 token。

GCG 的重要性在于，它把问题重新定义成：

- 用户原始请求保持不变。
- 只在后面追加一个可优化 suffix。
- 目标不是手写一个 scenario，而是最大化模型产生某类目标开头的概率。
- 通过 multi-prompt 和 multi-model 训练，让 suffix 不只对一个样本有效。

这就是它成为经典基线的原因：它让 jailbreak 研究从 anecdote 走向 optimizer + benchmark + transfer analysis。

## 2. 原论文结构地图

建议按这个顺序读原文：

- Abstract：先抓住三个关键词，automatic suffix、greedy gradient search、transferability。
- Figure 1：只看结构，不要沉迷示例文本。重点是一个 suffix 可以跨多个请求和多个模型触发失败。

- Section 1 Introduction: 看作者如何把 manual jailbreak 和 adversarial examples 两条线接起来。
- Section 2.1: 读 affirmative response objective。核心是为什么只优化响应开头，而不是完整危险回答。
- Equations 1-4: 读语言模型条件概率、target sequence negative log likelihood、suffix token 离散优化。
- Algorithm 1: 读 Greedy Coordinate Gradient。重点是 one-hot gradient -> top-k token candidates -> exact forward eval。
- Algorithm 2: 读 universal prompt optimization。重点是多 prompt 聚合、逐步加入 prompt、多模型同 tokenizer 聚合。
- Section 3: 读 AdvBench、ASR、white-box 结果、transfer 结果。
- Figure 6: 读 overfitting。优化 source model loss 太久，可能降低 black-box transfer。
- Section 5 和 6: 读 future work、adversarial training、ethics and broader impact。

### 3. 核心概念

#### Aligned LLM

这里的 alignment 是窄义的安全对齐：模型面对某些危险或不允许的请求时应该拒绝，而不是生成有害内容。它不等于完整的人类价值对齐。

#### Jailbreak

jailbreak 是让模型绕过自己的拒答策略，输出本应拒绝的内容。它可以是人工 prompt，也可以是自动搜索得到的 prompt。

#### Adversarial suffix

suffix 是追加在用户请求后的一串 token。论文关注的是：原始请求不改，只优化后缀。这样可以追问一个更干净的问题：是否存在一个通用后缀，让很多请求和很多模型都更容易失败。

#### Attack Success Rate, ASR

ASR 是成功率。对 exact string task，是模型是否输出目标字符串；对 behavior task，是模型是否尝试遵从不应遵从的行为。后者需要人工或规则判断，更接近 red-team eval。

#### Transferability

transfer 指一个在 source model 上优化出来的 suffix，能否在未参与优化的 target model 上也有效。论文最震撼的点不是 white-box 成功，而是迁移到不同模型、不同接口。

#### GCG

GCG 是 Greedy Coordinate Gradient。它不是直接在连续 embedding 里优化，也不是纯随机搜索。它先用 gradient 找每个 token 位置的候选替换，再用 forward pass 精确评估候选。

### 4. 威胁模型和任务形状

论文的输入结构可以抽象成：

```
system / chat template
user request r
adversarial suffix p
assistant response y
```

模型看到的是：

[chat prefix, user request  $r$ , suffix  $p$ , assistant marker]

攻击者能改的是 suffix  $p$ , 不改 request  $r$ , 不改 system prompt, 不改模型参数。

在 token 级别:

$x_{1:n}$  = full prompt tokens  
 $I$  = suffix token positions  
 $V$  = vocabulary size  
 $x_i$  = token id at position  $i$   
 $e_i$  = one-hot vector for token  $x_i$ , shape  $(V,)$

目标是让模型更可能输出一个 target response prefix。为了安全, 这里不写原文危险例子, 只写抽象形式:

target prefix  $y_{\text{star}}$ :  
an affirmative opening that mirrors the user's request

为什么只优化 response prefix?

- 完整回答有很多可能形式, 指定唯一完整答案太窄。
- 只优化第一个 token 又太弱, 可能让模型偏离原请求。
- 优化一个肯定式开头, 既给模型一个模式切换信号, 又保留原请求语义。

这是一种很重要的 red-team 设计: 目标函数不直接等于完整违规输出, 而是优化一个会让违规输出更可能发生的前缀状态。

## 5. 数学目标函数

语言模型给定 prefix tokens  $x_{1:n}$ , 预测下一个 token:

$p(x_{n+1} \mid x_{1:n})$

生成一段 target prefix  $y_{1:H}$  的概率是逐 token 条件概率乘积:

$p(y_{1:H} \mid x_{1:n})$   
= product over  $h$  from 1 to  $H$  of  $p(y_h \mid x_{1:n}, y_{1:h-1})$

论文优化 negative log likelihood:

$L(x_{1:n}) = -\log p(y_{\text{star}_{1:H}} \mid x_{1:n})$

suffix 优化问题是:

minimize  $L(x_{1:n})$   
subject to  $x_i$  in  $\{1, \dots, V\}$  for every  $i$  in suffix positions  $I$

关键难点:

- token 是离散的, 不能像图像像素一样直接连续更新。
- vocab 很大, suffix 长度  $l$  不小, 穷举所有替换不可行。
- 梯度是在 one-hot 或 embedding 连续化视角下算的, 只是候选提示, 不保证替换后真的变好。

GCG 的设计就是对这个难点做工程折中:

gradient:  
cheap way to rank promising token replacements

forward evaluation:

exact way to verify the replacement after discretization

coordinate greediness:

change one suffix token at a time

## 6. Algorithm 1: GCG 怎么工作

GCG 一步可以拆成四个动作:

current suffix  $p$ : length  $l$

loss  $L$ : target prefix NLL

1. Compute gradients

for every suffix position  $i$ :

grad <sub>$i$</sub>  = gradient of  $L$  with respect to one-hot token  $e_i$

shape:  $(V,)$

2. Candidate proposal

for every position  $i$ :

choose top- $k$  token ids with largest negative gradient

these are tokens predicted to reduce loss

3. Candidate batch evaluation

sample  $B$  candidate replacements from all positions

run actual forward passes

compute exact loss for each candidate suffix

4. Greedy accept

keep the candidate with minimum loss

ASCII 图:

suffix tokens  $p_1 \dots p_l$

|

v

one-hot relaxations  $e_1 \dots e_l$

|

v

backprop loss  $L$

|

v

gradients  $g_i: (V,)$  for each position

|

v

top- $k$  candidate token ids per position

|

v

sample/evaluate  $B$  candidate suffixes

|

v

accept lowest-loss replacement

GCG 和 AutoPrompt 的差异很小但很要命:

- AutoPrompt 通常先选一个位置, 再看这个位置的候选。
- GCG 对所有 suffix 位置都算 top-k, 再从全局候选里评估。
- 在相同 forward batch size 下, GCG 搜索空间覆盖更广。

这解释了为什么 Table 1 里 GCG 明显强于 AutoPrompt, 尤其在更难的 LLaMA-2-Chat 设置里。

## 7. Algorithm 2: universal multi-prompt and multi-model

如果只对一个请求优化一个 suffix, 很容易过拟合。论文真正想证明的是 universal attack:

```
one suffix p
works for many requests r_j
possibly across multiple source models M_s
```

Algorithm 2 做了三件事。

### 多 prompt 聚合

每个 prompt 有自己的 loss:

```
L_j(prompt_j + suffix)
```

优化时看 aggregate loss:

```
sum over active prompts j of L_j
```

候选 token 的梯度也聚合, 论文还提到会先把梯度裁剪到 unit norm 再聚合, 避免某个 prompt 支配搜索。

### curriculum: 逐步加入 prompt

一开始只优化第一个 prompt。当前 suffix 对已加入 prompt 都成功后, 再加入下一个 prompt。这样比一开始把所有 prompt 都扔进去更稳定。

```
active prompts = [prompt_1]
optimize suffix
if suffix works on active prompts:
    add prompt_2
repeat until all prompts are active
```

### 多模型聚合

如果多个 source model 使用同一 tokenizer, 那么 one-hot gradient 维度都是 V, 可以直接聚合。论文主要用 Vicuna-7B/13B, 也加入 Guanaco 变体, 生成更可迁移的 suffix。

关键直觉:

```
single prompt + single model:
    finds a local exploit for one case
```

```
many prompts + many models:
    forces suffix to rely on shared non-robust features
    -> more likely transfer
```

## 8. 和 soft prompt/embedding optimization 的区别

soft prompt 很容易优化，因为 embedding 是连续的。但 public chatbot 通常不允许你输入任意 embedding，只允许输入文本。因此 soft prompt 不一定能转回真实 token。

GCG 选择直接优化离散 token:

- 梯度只用来提出候选。
- 最终接受的是实际 token replacement。
- 每次候选都用真实 forward pass 评估。

这也是为什么它在真实接口威胁模型下更有意义。

## 9. 论文的 benchmark: AdvBench

论文设计了 AdvBench，包括两个设置。

### Harmful Strings

任务是让模型输出某些目标字符串。这个任务更接近 exact generation control，指标清晰，但也更窄。论文报告 exact match ASR 和 cross entropy loss。

### Harmful Behaviors

任务是让模型对不应遵从的行为产生尝试性遵从。这个任务更像真实 red-team，但判定更复杂，需要判断输出是否拒绝、规避、还是尝试执行。

为了安全，这份 guide 不复现数据集里的具体危险条目。你只需要掌握数据形状:

AdvBench item:

```
id
category
request_or_target
split: train or test
expected_safe_behavior: refusal / safe redirection
attack_result:
  model response
  label: success or failure
  optional loss
```

核心指标:

$ASR = \text{successful attack cases} / \text{total evaluated cases}$

对 universal suffix，还要区分:

- train ASR: 优化时见过的 behaviors。
- test ASR: held-out behaviors。

train/test 都高，才说明 suffix 不只是记住训练条目。

## 10. 实验证据链: white-box

Table 1 是 white-box 证据链的主表。论文比较 GBDA、PEZ、AutoPrompt、GCG。

Vicuna-7B:

- individual harmful strings: GBDA 0.0, PEZ 0.0, AutoPrompt 25.0, GCG 88.0。

- individual harmful behaviors: GBDA 4.0, PEZ 11.0, AutoPrompt 95.0, GCG 99.0。
- multiple harmful behaviors test ASR: GBDA 6.0, PEZ 3.0, AutoPrompt 98.0, GCG 98.0。

LLaMA-2-7B-Chat:

- individual harmful strings: GBDA 0.0, PEZ 0.0, AutoPrompt 3.0, GCG 57.0。
- individual harmful behaviors: GBDA 0.0, PEZ 0.0, AutoPrompt 45.0, GCG 56.0。
- multiple harmful behaviors test ASR: GBDA 0.0, PEZ 1.0, AutoPrompt 35.0, GCG 84.0。

这些结果支持三个结论:

- embedding/soft prompt 类方法在这个设置里不够稳定。
- AutoPrompt 已经有一定能力, 但 GCG 的 all-coordinate candidate search 更强。
- universal suffix 不是只对训练 behaviors 有效, 在 held-out behaviors 上也有效。

Figure 2 进一步说明 optimizer 过程:

- GCG 的 loss 下降更快。
- GCG 的 exact-match ASR 上升更明显。
- 这支持“GCG 不是偶然找到一个字符串, 而是优化器本身更有效”。

## 11. 实验证据链: black-box transfer

Section 3.2 更关键, 因为它测试 transfer。

训练方式:

- 用 25 个 behaviors。
- 用 Vicuna-7B 和 Vicuna-13B 优化 suffix。
- 另一个设置加入 Guanaco-7B 和 Guanaco-13B。
- 每次运行 500 steps, 生成多个 suffix。

测试方式:

- 在 open-source models 和 proprietary models 上测试。
- 使用 388 个 held-out behaviors。
- 比较 no attack、简单 affirmative baseline、GCG suffix、concatenation、ensemble。

Table 2 的 proprietary model 结果很重要:

- no attack baseline 几乎很低。
- 简单 affirmative baseline 也明显低。
- GCG optimized on Vicuna 在 GPT-3.5/GPT-4/PaLM-2 上有非平凡 ASR。
- Vicuna + Guanaco 的 suffix 提高了某些模型上的迁移。
- ensemble 在 GPT-3.5 达到 86.6, 在 GPT-4 达到 46.9, 在 Claude-1 达到 47.9, 在 PaLM-2 达到 66.0。
- Claude-2 明显更低, 论文报告 ensemble 为 2.1。

这些数字的意义不是鼓励攻击, 而是说明:

source model vulnerabilities can encode transferable features  
black-box safety cannot be assessed only by direct prompting  
model lineage/distillation may affect transfer

论文还指出一个有趣现象: Vicuna 来自 ChatGPT 输出数据, 可能让它和 GPT 系列之间存在更强 transfer。这呼应 adversarial examples 研究里的 transferability 和 distilled model 现象。



## 12. Figure 6: 过拟合和迁移

Figure 6 是很值得慢读的图。

左图显示:

- source-model GCG loss 前半程下降很快。
- black-box transfer ASR 前半程上升。
- 后半程 loss 继续优化或趋平, 但 transfer ASR 可能下降。

这说明 adversarial suffix 也会 overfit:

optimize too little:

suffix not strong enough

optimize enough:

captures shared vulnerable features

optimize too much:

specializes to source models

transfer may degrade

对学习者来说, 这是很重要的实验味道: 安全攻击不是 loss 越低越一定越能迁移。source objective 和 target risk 之间有 gap。

## 13. 伦理讨论和 disclosure

Section 6 明确承认这项研究有 misuse risk。作者的立场是:

- 技术并非不可发现, 已有类似方法和 manual jailbreak 传播。
- 更透明的披露能推动安全研究和防御。
- 论文发布前向 OpenAI、Google、Meta、Anthropic 做了 preliminary disclosure。
- 未来 LLM 接入 autonomous actions 后, 风险会更高。

我们在本仓库采用更保守的学习方式:

- 不保存真实攻击 suffix。
- 不保存危险完整输出。
- 本地代码只跑 mock target。
- 学习重点放在评测、优化结构和防御含义。

这也正是你用 AI agent 学这篇论文时要遵守的边界: 让 agent 解释机制、复现 toy optimizer、设计安全 eval, 而不是让它生成 payload。

## 14. 方法图: 从请求到 ASR

AdvBench requests

|

v

choose target prefix objective

|

v

initialize suffix tokens

|

v

```

GCG optimization on source models
  |
  v
candidate universal suffixes
  |
  +-----+
  |                                     |
  v                                     v
white-box eval on source models       black-box transfer eval
  |                                     |
  v                                     v
ASR / loss / train-test gap           ASR across unseen models
  |                                     |
  v                                     v
safety conclusion:
alignment needs adversarial evaluation,
not only natural prompting tests

```

## 15. 代码样例: safe toy GCG

本仓库新增了:

`learning/red-team-jailbreak/src/gcg_original_minimal.py`

它不调用真实 LLM, 不包含真实 suffix, 不优化危险目标。它只优化一个 harmless toy score:

```

def gcg_step(suffix, effects, top_k=2):
    one_hot = suffix_to_one_hot(suffix, effects.shape[1])
    one_hot.requires_grad_(True)
    old_loss = toy_loss_from_one_hot(one_hot, effects)
    old_loss.backward()

    candidates = []
    for pos in range(suffix.numel()):
        ranked = torch.topk(-one_hot.grad[pos], k=top_k).indices
        for token_id in ranked.tolist():
            trial = suffix.clone()
            trial[pos] = token_id
            loss = exact_loss(trial, effects)
            candidates.append((float(loss), pos, token_id, trial))

    best_loss, best_pos, best_token, best_suffix = min(
        candidates,
        key=lambda item: item[0],
    )
    return best_suffix

```

这段代码对应论文 Algorithm 1:

- `one_hot.grad[pos]` 对应 token-level gradient。
- `topk(-grad)` 对应 promising replacements。
- `exact_loss(trial, effects)` 对应 forward evaluation。

- `min(candidates)` 对应 greedy accept。

真实论文里 `effects` 不是手写矩阵，而是 LLM forward/backward 得到的 target-prefix loss。toy 版把复杂模型替换成可解释小矩阵，方便你学离散优化结构。

## 16. 和本仓库代码的连接

建议按这个顺序读：

1. `learning/red-team-jailbreak/src/common.py`
  - 看 `MockTarget`、`AttackResult`、`attack_success_rate`。
  - 理解 ASR 评测形状。
2. `learning/red-team-jailbreak/src/gcg_original_minimal.py`
  - 看安全 toy GCG。
  - 对应论文 Equations 3-5 和 Algorithm 1。
3. `learning/red-team-jailbreak/src/gcg_minimal.py`
  - 看 mock suffix search loop。
  - 对应“suffix appended to query”的流程，不代表真实梯度攻击。
4. `learning/red-team-jailbreak/src/jailbench_runner.py`
  - 看多方法 ASR 汇总。
  - 对应 `AdvBench/JailbreakBench` 风格评测。
5. `learning/red-team-jailbreak/src/red_team_matrix.py`
  - 看 target x method 的 ASR matrix。
  - 对应 red-team report card。

本地测试：

```
.\.venv\Scripts\python.exe learning\red-team-jailbreak\src\tests\test_redteam.py
```

## 17. 一个 30-60 分钟本地实验

实验目标：只用 harmless toy objective 理解 GCG 的“gradient proposes, forward verifies”。

步骤：

1. 打开 `learning/red-team-jailbreak/src/gcg_original_minimal.py`。
2. 运行：

```
.\.venv\Scripts\python.exe learning\red-team-jailbreak\src\gcg_original_minimal.py
```

3. 修改 `toy_token_effects` 中某个位置的最大分数。
4. 重新运行，看最终 suffix 是否换到新的最高分 token。
5. 把 `top_k` 从 3 改成 1，再改成 4，观察是否影响路径。

你应该能解释：

- 为什么 top-k 太小可能错过候选。
- 为什么 gradient 只负责候选排序，最终还要 forward evaluation。
- 为什么逐坐标 greedy search 不保证全局最优。
- 为什么多 prompt/multi model 能提高 universal transfer 的可能性。

## 18. 论文没有证明什么

这篇论文很强，但要注意边界：

- 它证明当时一组模型存在自动 adversarial suffix 风险，不证明所有未来模型都同样脆弱。
- ASR 对判定规则敏感，尤其是 harmful behavior 需要人工判断。
- 黑盒模型接口、过滤器、采样参数会影响结果。
- suffix transfer 的成因没有完全解释，distillation、训练数据、tokenizer、模型族都可能影响。
- adversarial training 是否能解决问题仍是开放问题。
- 论文展示的是 attack effectiveness，不是完整防御方案。

最重要的一点：这篇论文提醒我们，alignment 不能只靠“模型看起来会拒绝直接请求”来证明安全。直接请求只是最弱的评测。

## 19. 对今天安全学习的意义

这篇论文对现在仍然重要，原因有四个：

- 它定义了一个强自动红队基线，后续很多 jailbreak/defense paper 都需要和 GCG 对比。
- 它说明 prompt-level safety wrapper 很可能被优化器搜索到边界条件。
- 它推动了 HarmBench、JailbreakBench、red-team matrix 这类标准化评测。
- 它把 adversarial examples 的 transferability 思维带进了 aligned LLM。

对防御工程的启发：

- 要做 adversarial evaluation，而不是只做人工 smoke test。
- 要区分 direct prompt robustness 和 optimized suffix robustness。
- 要测 train/test behaviors，不只测已知攻击。
- 要测 transfer 和 ensemble，不只测单一 suffix。
- 要记录拒答、规避、安全改写、错误遵从之间的 label schema。

## 20. 用 AI agent 正确学习这篇论文

推荐你这样使用 agent：

1. 先让 agent 解释 Algorithm 1，但要求它只用 toy vocab。
2. 让 agent 根据你画的图检查张量形状：suffix\_len、vocab\_size、top\_k、batch\_size。
3. 让 agent 问你：“为什么 soft prompt 不等于真实接口 threat model？”
4. 让 agent 把 Table 1 转成证据链，而不是只报数字。
5. 让 agent 要求你解释 Figure 6 的 overfitting。
6. 让 agent 帮你改 toy\_token\_effects，但不要生成真实攻击文本。
7. 最后让 agent 让你闭卷复述：“GCG 为什么比 AutoPrompt 强？”

一个好的提示词：

我在学习 GCG 论文。请只使用 toy target 和 harmless vocabulary。

请按公式、Algorithm 1、Algorithm 2、Table 1、Table 2、Figure 6 的顺序考我。

不要生成真实 jailbreak suffix 或危险请求。

每次只问一个问题，等我回答后纠正我的机制理解。

最后要求我把答案映射到本仓库的 gcg\_original\_minimal.py。

## 21. 闭卷掌握检查

读完后你应该能回答：

- 为什么 manual jailbreak 不能替代系统性安全评测。
- adversarial suffix threat model 里，攻击者能改什么，不能改什么。
- 为什么论文优化 target response prefix，而不是完整输出。

- negative log likelihood objective 怎么写。
- one-hot gradient 的形状是什么，它为什么能给 token replacement 排序。
- GCG 为什么还需要 forward evaluation。
- GCG 和 AutoPrompt 的关键差别是什么。
- Algorithm 2 为什么要逐步加入 prompt。
- multi-model 优化为什么要求 tokenizer 对齐更方便。
- AdvBench 的 Harmful Strings 和 Harmful Behaviors 区别是什么。
- Table 1 证明了 GCG 的哪一层优势。
- Table 2 证明了 transfer 的哪一层风险。
- Figure 6 为什么说明 source loss 和 transfer ASR 不完全一致。
- 论文伦理讨论为什么重要。
- 本仓库哪些文件对应 toy GCG、ASR eval、red-team matrix。

## 22. 一句话总结

这篇论文的核心不是某个神奇后缀，而是：把 jailbreak 变成离散优化问题，用 token-level gradient 提候选、forward pass 验证候选。

再通过 multi-prompt 和 multi-model 训练，论文证明 aligned LLM 需要面对可迁移的自动化红队评测。